



Mẫu phần mềm trên nền tảng kiến trúc Microservices

Bộ môn HTTT/Viện CNTT&TT

2// GV. TS. Trần Văn An



Nội dung báo cáo

01

Bối cảnh phát triển phần mềm

02

Tổng quan về kiến trúc ứng dụng

03

Các thành phần công nghệ cốt lõi

04

Thiết kế và tích hợp hệ thống

05

Định hướng phát triển tương lai



01

Bối cảnh phát triển phần mềm

Gia tăng mạnh mẽ về khối lượng giao dịch trong ứng dụng hiện đại

Xử lý dữ liệu lớn hiệu quả

Các hệ thống phần mềm hiện đại phải xử lý lượng dữ liệu lớn, đòi hỏi tối ưu hóa hiệu suất và khả năng mở rộng để đáp ứng nhu cầu giao dịch ngày càng tăng cao.



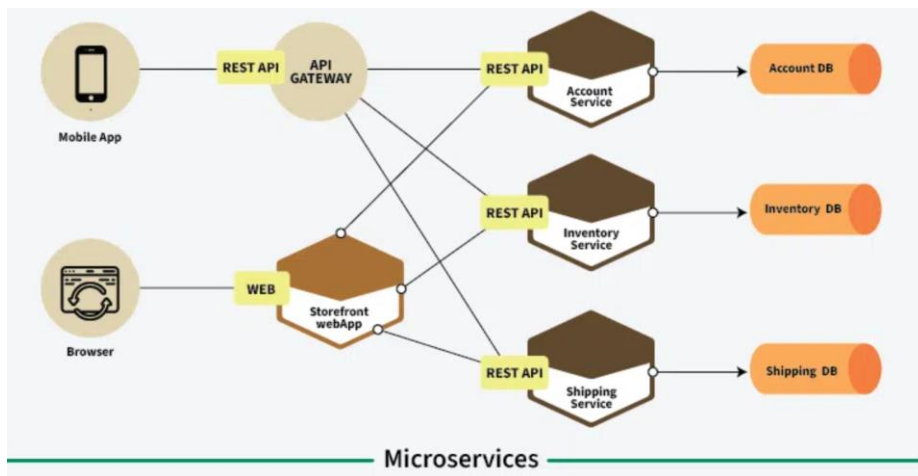
Sự đa dạng hóa các yêu cầu tương tác và nghiệp vụ của người dùng

Nhu cầu trải nghiệm đa nền tảng

Người dùng mong muốn ứng dụng hoạt động ổn định trên nhiều thiết bị, khiến hệ thống phải tương thích tốt với các môi trường khác nhau và duy trì giao diện nhất quán.

Yêu cầu nghiệp vụ ngày càng phức tạp

Các tổ chức thường xuyên bổ sung tính năng mới, đòi hỏi hệ thống phần mềm phải dễ dàng mở rộng và thích ứng với các thay đổi nghiệp vụ liên tục.



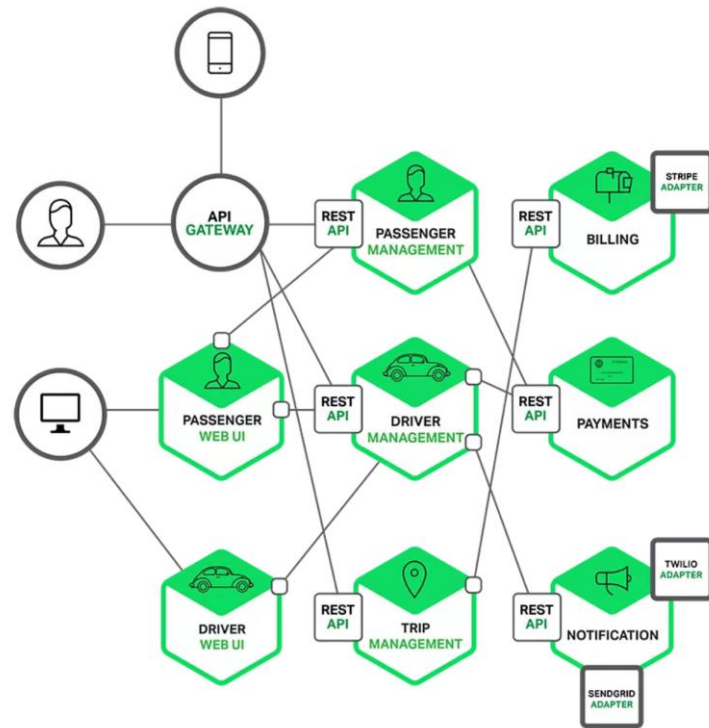
Xu hướng chuyển dịch sang mô hình dịch vụ vi mô trong kiến trúc hệ thống

Phân tách chức năng thành dịch vụ nhỏ

Kiến trúc microservices giúp hệ thống chia thành các dịch vụ độc lập, mỗi dịch vụ đảm nhiệm một chức năng, tăng khả năng phát triển và bảo trì riêng biệt.

Tối ưu hóa quy trình phát triển và triển khai

Mô hình dịch vụ vi mô cho phép đội ngũ phát triển cập nhật từng thành phần mà không ảnh hưởng đến toàn hệ thống, giúp triển khai nhanh và linh hoạt.





Tính linh hoạt và khả năng mở rộng cao trong kiến trúc ứng dụng hiện đại

Áp dụng nguyên tắc tách biệt mối quan tâm

Kiến trúc phân lớp giúp chia nhỏ hệ thống thành các tầng riêng biệt, mỗi tầng tập trung vào một nhiệm vụ, làm tăng khả năng bảo trì và mở rộng.

Hỗ trợ triển khai đa môi trường

Kiến trúc hiện đại cho phép hệ thống vận hành tốt trên nhiều hệ điều hành và nền tảng, từ máy chủ vật lý đến đám mây, nâng cao hiệu quả triển khai.

Kết hợp đa công nghệ tiên tiến

Việc tích hợp React, .NET Core và SQL Server cho phép xây dựng giải pháp linh hoạt, đáp ứng các tiêu chuẩn kỹ thuật và dễ dàng thay đổi công nghệ.

Khả năng thêm mới module nghiệp vụ dễ dàng

Hệ thống được thiết kế để dễ dàng tích hợp thêm các chức năng mới mà không cần thay đổi cấu trúc tổng thể, đảm bảo đáp ứng tốt nhu cầu phát triển lâu dài.



02

Tổng quan về kiến trúc ứng dụng

Giới thiệu kiến trúc Microservices và hệ thống phân tán hiện đại

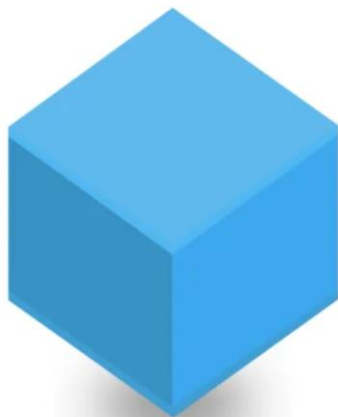
Khái niệm kiến trúc Microservices và phân tán

Kiến trúc microservices cho phép phân chia hệ thống thành các dịch vụ nhỏ độc lập, dễ mở rộng và bảo trì, phù hợp với yêu cầu hệ thống phân tán hiện đại.

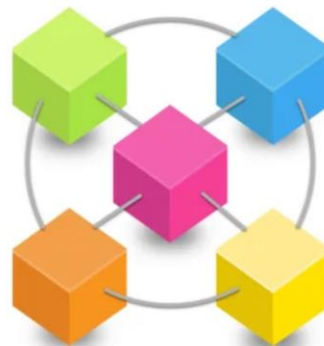
Xu hướng chuyển đổi từ mô hình tập trung sang phân tán

Chuyển dịch từ hệ thống monolithic truyền thống sang kiến trúc phân tán nhằm nâng cao khả năng mở rộng, tính linh hoạt và hiệu năng hệ thống.

Monolithic



Microservices



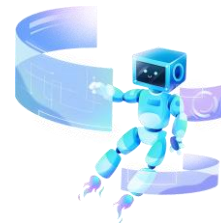


Yêu cầu và đặc điểm công nghệ hiện đại trong phát triển phần mềm

01

Sử dụng React cho giao diện người dùng

React hỗ trợ xây dựng UI tương tác cao, dựa trên thành phần, tối ưu hóa hiệu suất qua Virtual DOM và luồng dữ liệu một chiều.



02

.NET Core cho logic nghiệp vụ

Nền tảng đa nền tảng, mã nguồn mở, hỗ trợ phát triển dịch vụ web hiệu quả, phù hợp với kiến trúc microservices.

Nguyên tắc thiết kế phân lớp

Phân chia rõ các tầng: UI, API, dịch vụ, truy cập dữ liệu giúp hệ thống dễ bảo trì, mở rộng và kiểm thử.

03

SQL Server làm hệ quản trị dữ liệu

Hệ thống quản lý dữ liệu quan hệ mạnh mẽ, đảm bảo tính toàn vẹn, nhất quán và khả năng mở rộng của hệ thống dữ liệu.

Xu hướng phát triển và tích hợp công nghệ



Ứng dụng SPA và React

SPA cung cấp trải nghiệm mượt mà, React hỗ trợ xây dựng UI thành phần, tối ưu hiệu suất qua Virtual DOM và JSX.



Kiến trúc API và giao tiếp HTTP/RESTful

API dựa trên chuẩn RESTful, sử dụng HTTP phương thức rõ ràng, JSON làm định dạng dữ liệu, đảm bảo tính mở rộng và dễ tích hợp.



Quản lý trạng thái và xác thực người dùng

Sử dụng JWT để xác thực, bảo vệ API, nâng cao bảo mật và trải nghiệm người dùng.



Thiết kế hệ thống phân lớp và tích hợp công nghệ

01

Kiến trúc phân lớp rõ ràng

Chia thành các tầng: Controller, Service, Repository, Database, giúp tách biệt chức năng và dễ bảo trì.

02

Vai trò của các lớp Service và Repository

Service chứa logic nghiệp vụ, Repository trừu tượng hóa truy cập dữ liệu, tuân thủ nguyên tắc Separation of Concerns.

03

Quản lý mã nguồn và thành phần dùng chung

Sử dụng thư viện BaseCore.Common để chứa các thành phần hỗ trợ, giảm trùng lặp, tăng khả năng tái sử dụng và mở rộng.



03

Các thành phần công nghệ cốt lõi

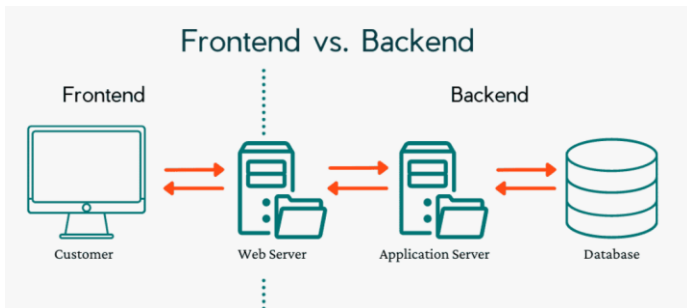
Giới thiệu kiến trúc phân tán sử dụng React, .NET Core và SQL Server

Mục tiêu của kiến trúc hệ thống phân tán

Hệ thống hướng tới khả năng mở rộng, hiệu năng cao và linh hoạt. Sử dụng kiến trúc phân tán để thích ứng với yêu cầu thương mại điện tử và các dịch vụ số, phân tách các chức năng thành các dịch vụ nhỏ, độc lập.

Chọn lựa công nghệ phù hợp

React cho giao diện, .NET Core cho lớp nghiệp vụ và SQL Server dựa trên các nguyên tắc tách biệt và tiện ích của mô hình phân lớp, nhằm đạt tiêu chuẩn kỹ thuật cao và dễ mở rộng.



Cấu trúc và các thành phần chính trong kiến trúc ứng dụng



React - Giao diện người dùng hiện đại

Thư viện JavaScript cho xây dựng UI dựa trên thành phần, tối ưu hiệu suất bằng Virtual DOM, luồng dữ liệu một chiều, và cú pháp JSX để sử dụng, phù hợp cho ứng dụng SPA tương tác cao.



.NET Core - Nền tảng phát triển đa nền tảng và hiệu suất cao

Hỗ trợ kiến trúc microservices, cung cấp Web API chuẩn REST, dễ kiểm thử, tích hợp Dependency Injection, phù hợp với môi trường phân tán và đa nền tảng.



SQL Server - Quản lý dữ liệu mạnh mẽ

Hệ quản trị quan hệ, hỗ trợ xử lý giao dịch ACID, công cụ phân tích, báo cáo, tích hợp dịch vụ dữ liệu, giữ vai trò trung tâm trong lưu trữ và xử lý dữ liệu.

Ưu điểm của từng thành phần công nghệ



React - Giao diện phản ứng nhanh, dễ mở rộng

Cơ chế Virtual DOM tối ưu hiệu suất, mô hình thành phần giúp tái sử dụng, phù hợp cho SPA với cập nhật dữ liệu liên tục, hỗ trợ kết xuất phía server và quản lý trạng thái hiệu quả.



.NET Core - Linh hoạt, đa nền tảng, hiệu suất cao

Hỗ trợ microservices, Web API, Dependency Injection, dễ kiểm thử và bảo trì, phù hợp với phát triển hệ thống phân tán quy mô lớn trong môi trường đa nền tảng.



SQL Server - Đảm bảo tính toàn vẹn và hiệu năng cao

Quản lý dữ liệu relations, tối ưu hóa bằng index, xử lý giao dịch ACID, tích hợp công cụ phân tích, báo cáo, phù hợp cho hệ thống doanh nghiệp lớn.



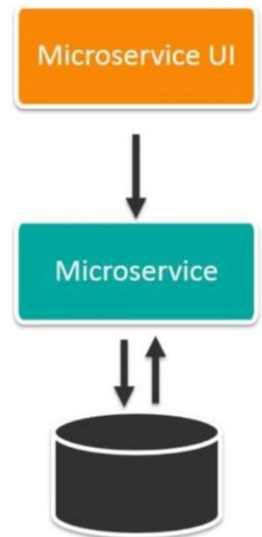
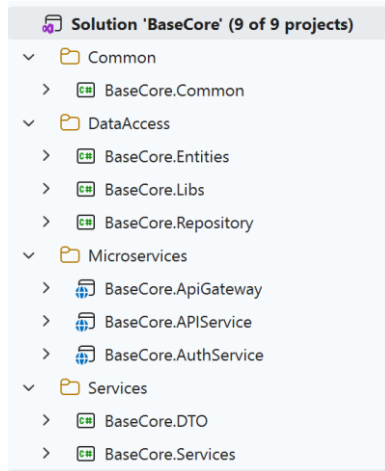
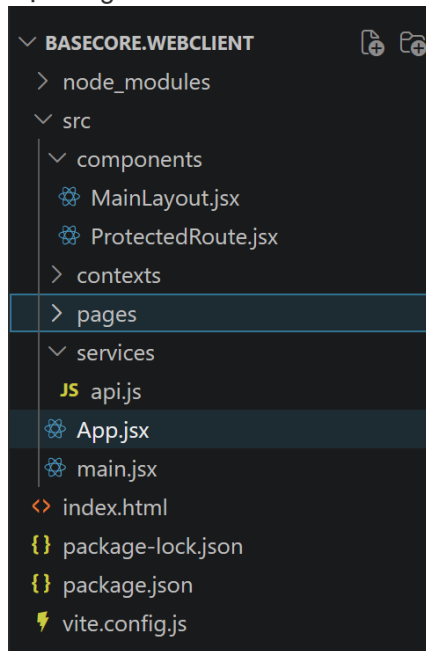
04

Thiết kế và tích hợp hệ thống

Mô hình kiến trúc phân lớp trong hệ thống phân tán hiện đại

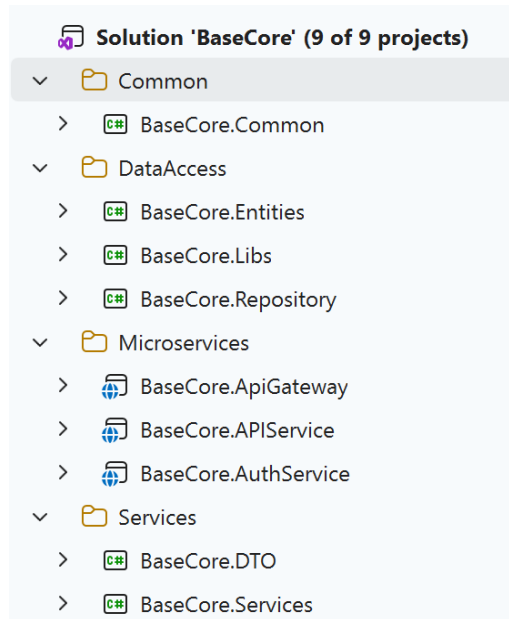
Phân chia tầng chức năng rõ ràng

Kiến trúc phân lớp tạo ra các tầng riêng biệt như Frontend, API, Service, Repository và Database, giúp kiểm soát luồng xử lý và nâng cao khả năng bảo trì hệ thống.





Tách biệt mối quan tâm và tối ưu hóa trách nhiệm các thành phần



Phân vai trò Controller, Service, Repository

Controller tiếp nhận yêu cầu, Service xử lý nghiệp vụ, Repository thao tác dữ liệu, hạn chế trùng lặp và tăng tính module hóa cho toàn hệ thống.

Sử dụng Data Transfer Object (DTO)

DTO giúp vận chuyển dữ liệu giữa các tầng mà không lộ thông tin nội tại của Entity, đảm bảo bảo mật và kiểm soát dữ liệu chặt chẽ.

Triển khai nguyên tắc Separation of Concerns

Tách biệt nhiệm vụ của từng tầng giúp mã nguồn dễ bảo trì, dễ mở rộng và thuận tiện kiểm thử đơn vị cho từng thành phần.

Lợi ích kiểm thử và thay đổi công nghệ

Việc tách biệt các tầng cho phép kiểm thử độc lập từng module và dễ dàng chuyển đổi công nghệ mà không ảnh hưởng toàn bộ hệ thống.

Quản lý cấu trúc mã nguồn và khả năng mở rộng hệ thống



Tổ chức thành phần dùng chung hợp lý

Thư viện BaseCore.Common tập trung các chức năng chia sẻ như xác thực, tiện ích file, WebSocket, giúp giảm trùng lặp và tăng khả năng tái sử dụng.

Phân tách logic nghiệp vụ khỏi Controller

Các Service đảm nhiệm xử lý nghiệp vụ, Controller chỉ tiếp nhận và chuyển tiếp, tăng tính rõ ràng, dễ bảo trì và kiểm thử cho hệ thống.



Áp dụng kỹ thuật thiết kế hướng đối tượng

Sử dụng Interface, kế thừa, đa hình và đóng gói giúp hệ thống linh hoạt, dễ mở rộng, đáp ứng thay đổi nghiệp vụ nhanh chóng.

Tối ưu hóa truy cập dữ liệu qua Repository

Repository đảm nhận giao tiếp với Database, cho phép thay đổi công nghệ lưu trữ không ảnh hưởng đến logic nghiệp vụ và các tầng trên.



Cơ chế giao tiếp hiệu quả giữa Frontend React và Backend .NET Core

Sử dụng API Gateway và chuẩn RESTful

API Gateway là điểm truy cập duy nhất, điều phối yêu cầu từ Frontend đến từng microservice, áp dụng RESTful chuẩn hóa giao tiếp qua HTTP.

Quản lý xác thực và trạng thái bằng JWT

JWT dùng cho xác thực, lưu trữ token phía client, kiểm soát truy cập API; đồng thời React quản lý trạng thái để giao diện luôn đồng bộ.

```
// JWT Authentication
var key = Encoding.ASCII.GetBytes(builder.Configuration["Jwt:SecretKey"] ?? "YourSecretKeyForAuthenticationShouldBel
builder.Services.AddAuthentication(x =>
{
    x.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    x.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(x =>
{
    x.RequireHttpsMetadata = false;
    x.SaveToken = true;
    x.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(key),
        ValidateIssuer = false,
        ValidateAudience = false
    };
});
```



05

Định hướng phát triển tương lai



Định hướng mở rộng tính năng và nâng cao bảo mật hệ thống ứng dụng phân tán

01

Mở rộng chức năng nghiệp vụ

Việc tăng cường các module nghiệp vụ mới giúp hệ thống thích ứng với nhu cầu thị trường, đảm bảo khả năng tích hợp linh hoạt mà không ảnh hưởng tới các dịch vụ hiện hữu.

02

Nâng cao bảo mật xác thực

Áp dụng các kỹ thuật xác thực nâng cao, như đa lớp xác thực và kiểm soát phiên, nhằm tăng cường bảo vệ dữ liệu người dùng và hạn chế truy cập trái phép hiệu quả.

03

Giảm phụ thuộc giữa dịch vụ

Tích hợp bộ môi giới tin nhắn để giảm kết nối trực tiếp giữa các microservice, giúp hệ thống duy trì ổn định và mở rộng quy mô dễ dàng khi phát sinh dịch vụ mới.

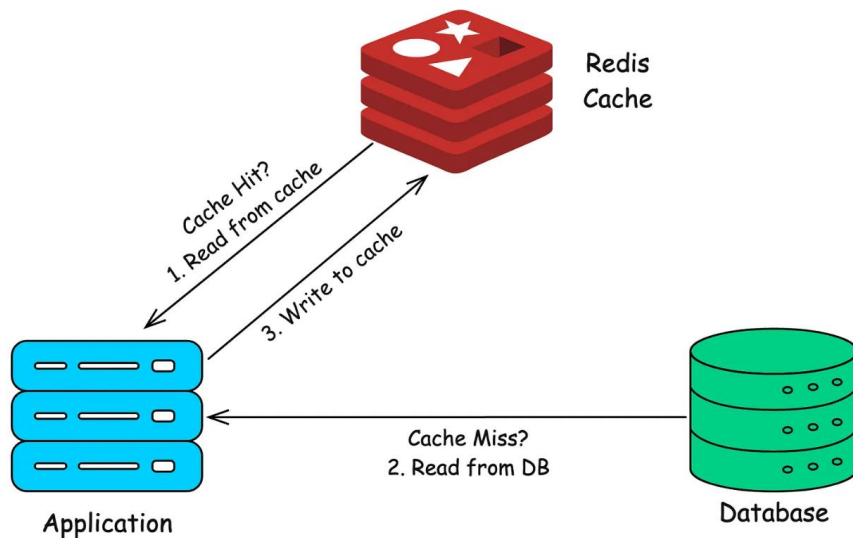
Tối ưu hóa hiệu suất truy xuất và quản lý dữ liệu trong hệ thống doanh nghiệp

Tích hợp giải pháp Caching hiện đại

Ứng dụng Redis Cache hoặc giải pháp tương tự để lưu tạm dữ liệu, giảm tải truy vấn trực tiếp lên SQL Server và tăng tốc độ phản hồi cho các thao tác thường xuyên.

Tối ưu hóa cấu trúc chỉ mục dữ liệu

Thiết kế hợp lý hệ thống chỉ mục và chuẩn hóa bảng giúp cải thiện hiệu năng truy vấn, đảm bảo xử lý tốt ngay cả khi dung lượng dữ liệu tăng mạnh.



Cải thiện quy trình phát triển và quản lý mã nguồn trong dự án phần mềm doanh nghiệp



Tối ưu hóa module dùng chung

Tăng cường sử dụng thư viện tiện ích chung để giảm trùng lặp mã nguồn, nâng cao khả năng tái sử dụng và đơn giản hóa bảo trì toàn dự án.



Chuẩn hóa quy trình kiểm thử tự động

Xây dựng quy trình kiểm thử tự động cho từng lớp Service và Repository, giúp phát hiện lỗi sớm, đảm bảo chất lượng và độ ổn định khi mở rộng hệ thống.



Đẩy mạnh áp dụng thiết kế hướng đối tượng

Áp dụng triệt để các nguyên tắc thiết kế hướng đối tượng giúp hệ thống dễ mở rộng, dễ kiểm thử và linh hoạt khi tích hợp công nghệ mới hoặc thay đổi yêu cầu nghiệp vụ.



Xin trân thành cảm ơn thủ trưởng và các đồng chí!

BM HTTT/Viện CNTT & TT

2// GV. TS. Trần Văn An